

NPTEL Week 7

ASSIGNMENT 7 SOLUTIONS & EXPLANATIONS

PROGRAMMING SOLUTIONS

1. Attraction Ratings Directory

Builds a dictionary to map attraction names to their ratings, allowing for quick retrieval or returning -1 if absent.

// PYTHON

```
n = int(input())
attractions = {}

for _ in range(n):
    line = input().split()
    name = " ".join(line[:-1])
    rating = line[-1]
    attractions[name] = rating

target = input()
print(attractions.get(target, -1), end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
    el.focus();
    document.execCommand('selectAll', false, null);
    document.execCommand('insertText', false, `n = int(input())
attractions = {}

for _ in range(n):
    line = input().split()
    name = " ".join(line[:-1])
    rating = line[-1]
    attractions[name] = rating

target = input()
print(attractions.get(target, -1), end='')`);
}
```

2. Landmark Linear Search

Parses a list of landmarks and executes a sequential linear search to return the index of the target location.

// PYTHON

```
n = int(input())
landmarks = input().split()
target = input().strip()

try:
    print(landmarks.index(target), end='')
except ValueError:
    print(-1, end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
    el.focus();
    document.execCommand('selectAll', false, null);
    document.execCommand('insertText', false, `n = int(input())
landmarks = input().split()
target = input().strip()

try:
    print(landmarks.index(target), end='')
except ValueError:
    print(-1, end='')`);
}
```

3. Second Highest Value

Sorts an array of space-separated integers in ascending order to extract and return the second highest value.

// PYTHON

```
arr = list(map(int, input().split()))
arr.sort()
print(arr[-2], end='')
```

// CONSOLE INJECTION

```
const el = document.querySelector('textarea.inputarea, textarea');
if (el) {
    el.focus();
    document.execCommand('selectAll', false, null);
    document.execCommand('insertText', false, `arr = list(map(int, input().split()))
arr.sort()
print(arr[-2], end='')`);
}
```

MCQ & MSQ SOLUTIONS WITH EXPLANATIONS

Scenario 1: Drone Surveillance Route

Q01 **k and l**

Explanation: The variable `k = 0` tracks the starting row index, and `l = 0` tracks the starting column index for the current spiral boundary.

Q02 **It moves to the next row after printing the top row.**

Explanation: `k += 1` increments the starting row boundary, shrinking the grid downwards after the top row elements have been visited.

Q03 **It prints the last column from top to bottom.**

Explanation: The loop `for i in range(k, m):` iterates downward using index `n - 1`, which points to the rightmost unvisited column.

Q04 **1 2 3 4**

Explanation: Assuming a standard 4x4 matrix indexed 1 to 16, the very first operation is printing the top row from left to right.

Q05 **Selected Options:** MSQ

—The variables `k`, `l`, `m`, and `n` define the current boundaries of the unvisited portion of the matrix.

—After each side of the spiral is printed, the corresponding boundary is updated.

Explanation: These statements correctly describe how a 2D spiral traversal manages boundary constraints (top/bottom/left/right limits).

Scenario 2: Mini Snake and Ladder Tournament

Q06 **if position + roll <= target:**

Explanation: This condition guarantees the player does not exceed the target position (25), effectively ignoring excessive rolls.

Q07 **The player moves to position 7.**

Explanation: The script indicates that landing on position 16 triggers a snake, immediately modifying the position back down to 7.

Q08 **It checks whether the player has reached the target position.**

Explanation: Line 21 (`if position == target:`) serves as the win-condition check for completing the game.

Q09 **break**

Explanation: The `break` statement abruptly stops the execution of the dice roll loop when the win condition is met.

Q10 **Selected Options:** MSQ

—A move is ignored if it takes the player beyond position 25.

—Landing on a snake or ladder changes the player's position immediately.

Explanation: Position boundary condition and the `if/elif` chain for movement mechanics correctly reflect standard Snakes & Ladders rules.

Scenario 3: Food Delivery Pickup Locations

Q11 It creates a Google map centered at the given location.

Explanation: The `gmpplot.GoogleMapPlotter(...)` call instantiates the base map object using a specified latitude and longitude origin point.

Q12 `gmap.marker(lat, lon, title=name)`

Explanation: This specific method call plots a location pin on the instantiated map using coordinates.

Q13 The restaurant name

Explanation: The `title=name` parameter maps the tuple string value (e.g., "Spice Garden") to the marker's hover-text label.

Q14 It saves the map as an HTML file.

Explanation: The `gmap.draw(...)` function compiles the Google Maps JavaScript and marker data, outputting an interactive HTML document.

Q15 Selected Options: MSQ

—The markers are added using the latitude and longitude values.

—The program iterates through all restaurants using a for loop.

Explanation: Unpacking tuples (`name, lat, lon`) in a for loop dynamically passes spatial data to the plotter.

Case Study: Smart Heritage City Explorer

Q16 Linear Search

Explanation: Unordered arrays require iterating through elements sequentially ($O(n)$ time complexity) since binary search cannot be applied.

Q17 To inspect every location by moving from the outer boundary towards the center.

Explanation: The drone follows a boundary-contraction path algorithm, which structurally mirrors a spiral matrix algorithm.

Q18 Latitude and Longitude

Explanation: A GIS mapping utility fundamentally relies on geometric point data (lat/lon) to position visual markers.

Q19 The move is ignored.

Explanation: Rolling higher than the exact needed increment triggers the bounds safety check, meaning the player remains in place.

Q20 Selected Options: MSQ

—A dictionary is suitable for storing multiple details about each attraction.

—Spiral traversal is a suitable approach for visiting every location in a matrix.

Explanation: Dictionaries efficiently handle key-value metadata lookups ($O(1)$), while spiral traversal ensures complete 2D area coverage.